

بوابة الخدمات اللوجستية العكسية

Reverse Logistics Portal

مشروع إدارة عمليات الإرجاع والاستبدال بنظام Java Swing

نظرة عامة على المشروع

ما هي اللوجستيات العكسية؟

هي كافة العمليات المتعلقة بإرجاع المنتجات من العميل النهائي إلى الشركة، بما يشمل الاستبدال ورصد المبالغ المستردة.

أهداف النظام الأساسية:

- ✓ أتمتة طلبات الإرجاع (Returns).
- ✓ تنظيم عمليات الاستبدال (Exchanges).
- ✓ إدارة رد المبالغ المالية (Refunds) باحترافية.

البيئة التقنية

</> اللغة: Java (Backend & Logic)

الواجهات: Java Swing (GUI)

الأنماط: Design Patterns (Architecture)

مكونات وشاشات النظام



Admin Control

مراجعة الطلبات، اتخاذ قرار القبول/الرفض، ومعالجة المبالغ.



Customer Interface

واجهة سهلة للعميل لاختيار المنتج، تحديد السبب، ورفع الطلب.



Dashboard

إحصائيات كاملة عن إجمالي الطلبات، الحالات، والمبالغ المستردة.



Settings

إدارة بيانات المؤسسة، سياسات الاسترجاع، والمدد الزمنية.



Notifications

تحديثات حية (Live Feed) لكل العمليات التي تتم في النظام.

هندسة التصميم (UML)

تطبيق أنماط التصميم (Design Patterns)

تم الاعتماد على 3 أنماط تصميم رئيسية لحل مشاكل برمجية واقعية وضمان كفاءة الهيكل البنائي للمشروع.

1. نمط الاستراتيجية (Strategy Pattern)

✂ المشكلة البرمجية:

تعدد طرق رد الأموال يؤدي لتضخم كود `if-else`، مما يجعل النظام غير قابل للتوسعة (Hard to scale) عند إضافة طرق دفع جديدة مستقبلاً.

💡 الحل الهندسي:

تغليف خوارزميات الدفع في فئات مستقلة (Encapsulation) تنفذ واجهة موحدة، ويتم اختيارها ديناميكياً في وقت التشغيل (Runtime).

الاستراتيجيات المحققة بالنظام:

- ✓ **BankTransferRefund**: تحويل بنكي لحساب العميل.
- ✓ **StoreCreditRefund**: إضافة رصيد للمتجر مع بونص إضافي.
- ✓ **OriginalPaymentRefund**: إعادة الأموال للبطاقة الأصلية.

مخطط نمط الاستراتيجية المطابق تماماً (UML)

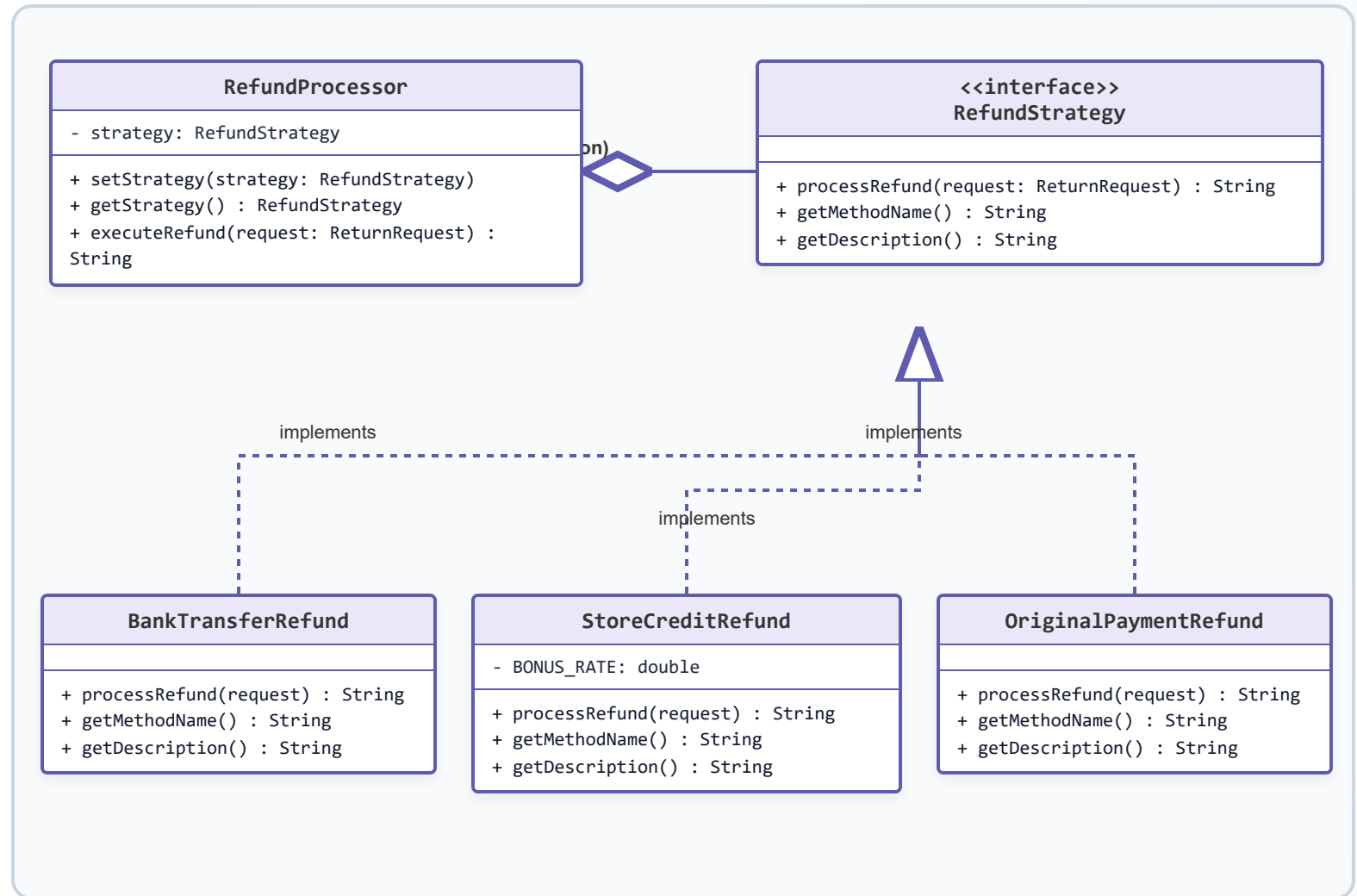
التحليل الهيكلي:

✓ **روابط ثابتة:** تم ربط العلاقات بالـ SVG

لضمان توجيه دقيق للأسهم مهما
اختلف حجم العرض.

✓ **تطابق تام:** يطابق الكود المخططات

الهندسية المرفقة من قبلكم في
تفاصيل الدوال والبارامترات.



2. نمط المراقب (Observer Pattern)

❌ المشكلة البرمجية:

ارتباط كود تغيير حالة الطلب مباشرة بكود التنبيهات وتحديث الواجهات يسبب "اقتران شديد" (Tight Coupling)، مما يمنع إضافة مراقبين جدد دون تعديل الكود الرئيسي.

١ الحل الهندسي:

فصل الكائنات المهتمة بالحالة وجعلها ترث من واجهة مراقب موحدة، ليقوم الكائن الأساسي ببث التحديثات تلقائياً عبر آلية المشتركين.



تحديث متزامن فور التغيير:

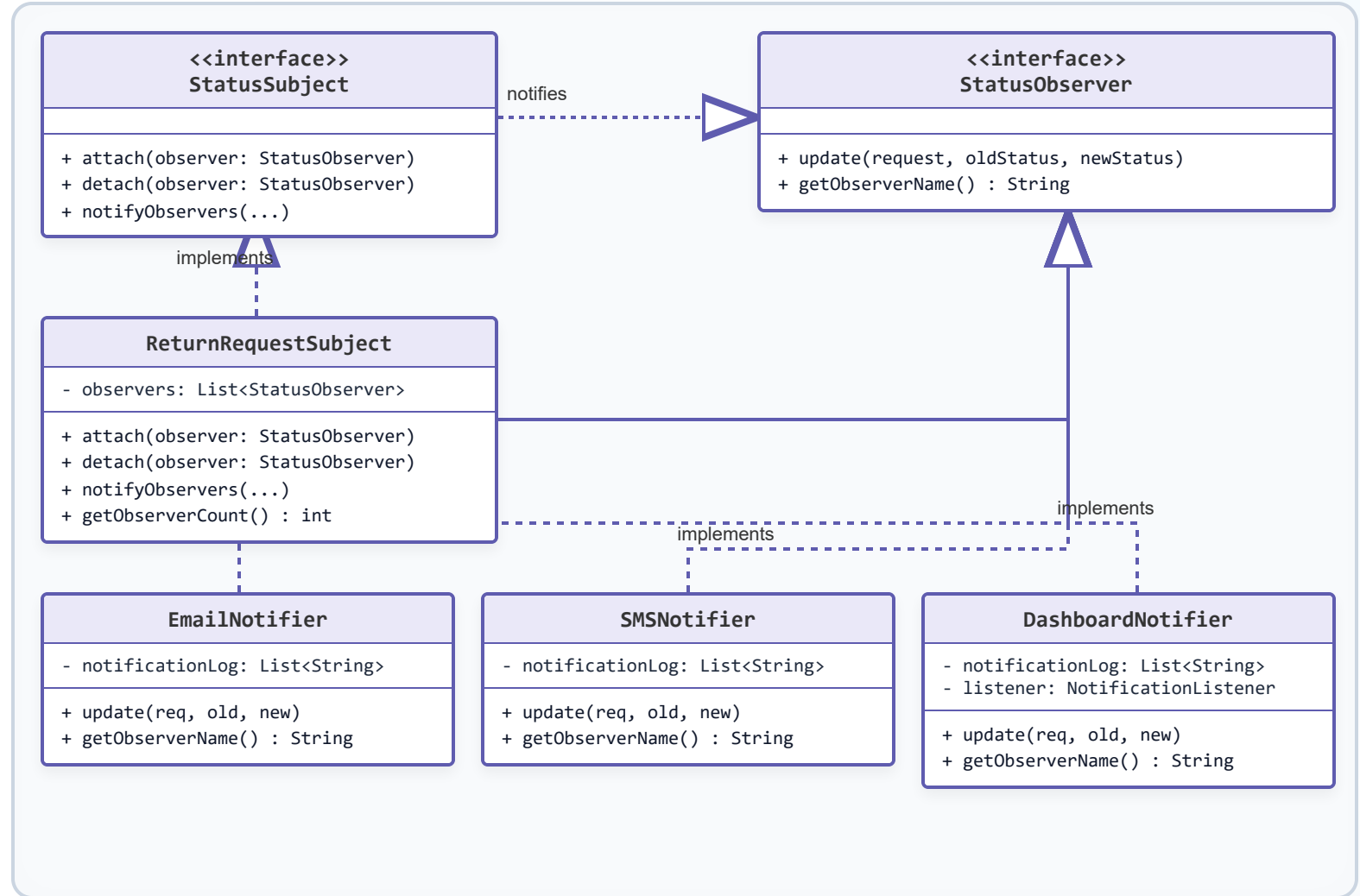
بمجرد الموافقة على طلب الإرجاع، يتم تحديث جدول Swing، وإرسال بريد إلكتروني، وتنبيه الـ SMS في نفس اللحظة برمجياً.

مخطط نمط المراقب المطابق تماماً (UML)

دقة بث البيانات:

✓ ربط SVG هندسي: الخطوط والأسماء تتبع إحداثيات الكلاسات بدقة بكسل ثابتة ومستقرة تماماً.

✓ تطابق التسميات: الهيكل يعكس تماماً أسماء ومحتويات صور الجافا المرفقة.



3. نمط السنجلتون (Singleton Pattern)

ConfigurationManager

ضمان وجود نسخة واحدة فقط من كود الإعدادات تشاركها كل واجهات وهياكل النظام، لمنع تضارب إعدادات المدد والنسب وتوفير الذاكرة العشوائية.

```
private static volatile ConfigurationManager instance;  
private ConfigurationManager() { /* حظر الإنشاء الخارجي */ }  
public static ConfigurationManager getInstance() { ...  
}
```

🔒 تم حماية النسخة بآلية **Double-Checked Locking** لضمان أمان خيوط المعالجة (Thread-Safety).

1

Instance الوحيد

إدارة البيانات عبر الـ Singleton

البيانات الموحدة والمشاركة برمجياً عبر الـ Instance الوحيد:

المتغير المدار (Attribute)	القيمة الافتراضية	الأهمية التقنية والربط البرمجي
اسم الشركة (CompanyName)	Reverse Logistics Portal .Co	التوحيد التلقائي في كافة الفواتير والتقارير والواجهات الرسومية لـ Swing
مدة الاسترجاع الصالحة (ReturnWindow)	30 يوماً	تحديد منطق التحقق (Logic Check) لرفض أو قبول الطلبات بناءً على التواريخ
حافز رصيد المتجر (BonusRate)	0.10 (10%)	يتم استدعاؤها مباشرة في كلاس StoreCreditRefund لحساب القيمة الإضافية للمرتجع

شكراً لكم!

هل لديكم أي أسئلة أو استفسارات للمناقشة؟

